

what is node js ----->>>

node js is not a language. Node js is async language

this is a Server run time Environment (run JavaScript on server environment)

Node js can connect with database

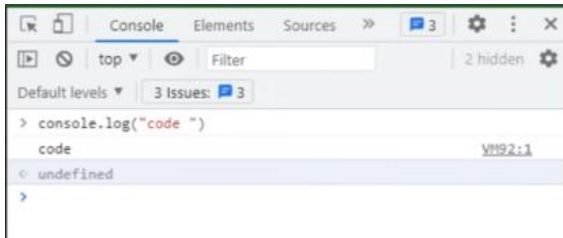
Code and Syntax is very similar to JavaScript, But not exact same

Node js free, open source

Node js use Chrome's V8 engine to execute code

Node js Version check ----->>> **node -v**

Npm js Version check ----->>> **npm -v**



With this example result shows undefined because with console.log only print the message not return anythings

```
> var a = 50
< undefined
> var b = 40
< undefined
> c = a+b
< 90
>
```

Here at the last equation result shows without undefined **because c is return the plus value of a and b**

Run node js from folder: ----->>>

Create a folder.

In side folder create index.js file.



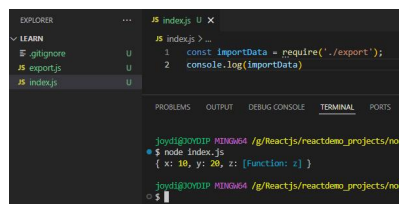
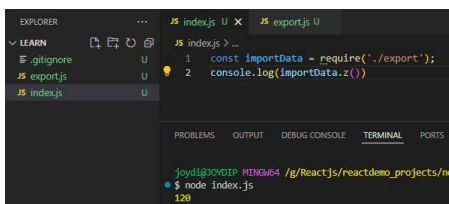
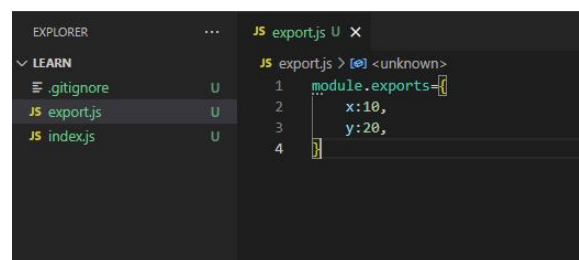
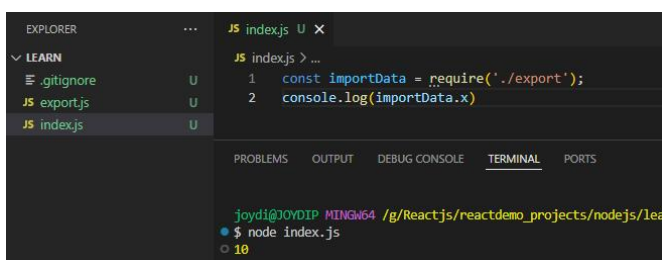
Difference between browser console and node js console ----->>>

Both console are different

browser console is a function of browser

node js console is a internal module of node js because node is not run in browser

Import file in node js



What is filter: -----

Filter is a function which helps to find a specific data from a array. It always works with array data only.

```
JS filter.js U X
JS filter.js > ...
1 const arr = [2,3,4,5,6,8,9,23];
2 arr.filter((item)=>{
3   console.log(item)
4 })
```

```
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects
$ node filter.js
2
3
4
5
6
8
9
23
```

```
JS filter.js U X
JS filter.js > ...
1 const arr = [2,3,4,5,6,8,9,23];
2 let filterResult = arr.filter((item)=>{
3   return item===5
4 })
5 console.log(filterResult);
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$ node filter.js
[ 5 ]
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$
```

```
JS filter.js U X
JS filter.js > filterResult > arr.filter() callback
1 const arr = [2,3,4,5,6,8,9,23];
2 let filterResult = arr.filter((item)=>{
3   return item > 5
4 })
5 console.log(filterResult);
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$ node filter.js
[ 6, 8, 9, 23 ]
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$
```

Types of module in Node JS -----

1. Core Module > 1.1> Global Module, 1.2> Non Global Module
2. External Module

In Node.js, **core modules** are built-in modules that come pre-packaged with the Node.js runtime. They provide essential functionality for various operations, such as working with files, streams, HTTP servers, cryptography, and more. These modules are directly available without requiring any additional installation or setup.

Characteristics of Core Modules

- ◆ **Pre-installed:** They are bundled with Node.js and don't require installation through npm.
- ◆ **Lightweight:** Optimized for performance and directly integrated with Node.js.
- ◆ **Always Available:** You can access them by simply using the `require()` function.

List of Node.js Core Modules: Other notable core modules include:

- ◆ **Buffer:** For working with binary data.
- ◆ **Stream:** For handling streaming data.
- ◆ **URL:** For URL resolution and parsing.
- ◆ **Timers:** For managing timeouts and intervals.
- ◆ **Zlib:** For compression and decompression.

Why Use Core Modules?

- ◆ No need to install additional dependencies.
- ◆ Highly efficient and optimized for Node.js.
- ◆ Reduce external dependencies in your project.

Global modules -----

In Node.js, global modules refer to Globally Installed Modules. These are modules installed system-wide and can be accessed from any project on your machine. They are not tied to a specific project directory.

To install a module globally, use the `-g` flag with npm:

Example: `npm install -g <module_name>`

Make a basic server and output on server: -----

In Node.js, the **http module** is a **core module** used to create HTTP servers and clients. It provides essential functionalities to interact with the **HTTP protocol**, enabling Node.js applications to handle web requests and responses.

Key Features of the http Module

- ◆ **Core Module:** Pre-installed with Node.js, no need to install it separately.
- ◆ **Web Server Creation:** You can create an HTTP server to handle client requests.
- ◆ **HTTP Client:** Allows making HTTP requests to external servers.
- ◆ **Customizable:** Offers flexibility to define how the server handles incoming requests and sends responses.

Basic Usage of the http Module

```
JS basic_server.js U X
JS basic_server.js > server.listen('4300') callback
1  const http = require('http');
2
3  const server = http.createServer((req, resp) => {
4    resp.write("This is a simple created server");
5    resp.end();
6  });
7
8  server.listen('4300', () => {
9    console.log("server is running on port: 4300");
10 });
11
12
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/
$ node basic_server.js
server is running on port: 4300
```

```
← → ↺ ⓘ localhost:4300
⌵ | 🌐 Discord | Friends | 📧 Gmail | 📁 All Tools
```

This is a simple created server

what is package.json -----

In Node.js, **package.json** is a file that provides metadata about a project or module. It serves as the central configuration file for a Node.js project and includes information about the project, its dependencies, scripts, and other settings.

Creating a package.json File

create a package.json file using the following command: **npm init**

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

Customize Package.json

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": ["nodejs", "example", "app"],
  "author": "author name",
  "license": "ISC"
}
```

```
"scripts": {
  "start": "node index.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

With this above start key we can run code by command: **npm start**

Create a Simple API: -----

```
const http = require("http");
http.createServer((req, resp)=>{
  resp.writeHead(200, { 'content-type' : 'application/json' });
  resp.write(JSON.stringify({ "name": "nodejs", "type": "javascript" }));
  resp.end();
}).listen(5000);
```

Pretty-print ☐

```
{"name":"nodejs","type":"javascript"}
```

Get Simple API Data from other file : -----

```
JS simple_api_datajs U X
JS simple_api_datajs > ...
1  const data = [
2    { "name": "nodejs", "type": "javascript" },
3    { "name": "reactjs", "type": "javascript" },
4    { "name": "angular", "type": "javascript" },
5  ];
6
7  module.exports = data;
```

```
JS simple_apijs U X
JS simple_apijs > http.createServer() callback
1  const http = require("http");
2  http.createServer((req, resp)=>{
3    resp.writeHead(200, { 'content-type' : 'application/json' });
4    resp.write(JSON.stringify(data));
5    resp.end();
6  }).listen(5000);
7
```

Pretty-print ☐

```
[{"name":"nodejs","type":"javascript"}, {"name":"reactjs","type":"javascript"}, {"name":"angular","type":"javascript"}]
```

Get input from command line : -----

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node command_line_get_input.js
[
  'C:\\Program Files\\nodejs\\node.exe',
  'G:\\Reactjs\\reactdemo_projects\\nodejs\\learn\\command_line_get_input.js'
]
```

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node command_line_get_input.js hello hi
[
  'C:\\Program Files\\nodejs\\node.exe',
  'G:\\Reactjs\\reactdemo_projects\\nodejs\\learn\\command_line_get_input.js',
  'hello',
  'hi'
]
```

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv[2]);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/
$ node command_line_get_input.js hello hi
hello
```

Create a file with text using fs core module and process.argv -----

```
JS command_line_get_input.js M X
JS command_line_get_input.js > ...
6
7 // create file with process
8 const fs = require("fs");
9 const input = process.argv;
10 fs.writeFileSync(input[2], input[3] ); // input[2] => file name, input[3] => text
11
12
13 You, 12 seconds ago • Uncommitted changes

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node command_line_get_input.js test.txt 'simple text write in file using FS and process.argv'

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
○ $
```

```
JS command_line_get_input.js M test.txt U X
test.txt
1 simple text write in file using FS and process.argv
```

For Create a file -----

`fs.writeFileSync`

For remove a file -----

`fs.unlinkSync('filename')`

Add and remove file using fs module and process -----

```
JS command_line_get_input.js M ●
JS command_line_get_input.js > ...
5
6
7 // add and delete file with process
8 const fs = require("fs");
9 const input = process.argv;
10 if(input[2] === "add"){
11     fs.writeFileSync(input[3], input[4]);
12 }else if(input[2] === "delete"){
13     fs.unlinkSync(input[3]);
14 }else{
15     console.log("enter value")
16 }
17
18 You, 2 minutes ago • Uncommitted change

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node command_line_get_input.js add add.txt 'simple text write in file using FS and process.argv'

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node command_line_get_input.js delete add.txt

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
○ $
```


Get folder path: -----

```
JS folder_path.js U X
JS folder_path.js > [0] dirpath
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname)
4  console.log(dirpath);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js
G:\Reactjs\reactdemo_projects\nodejs\learn
```

```
JS folder_path.js U X
JS folder_path.js > [0] dirpath
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  console.log(dirpath);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js
G:\Reactjs\reactdemo_projects\nodejs\learn

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js
G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder
```

Create file in inner folder of project folder: -----

```
EXPLORER  ...  JS folder_path.js U X
└─ LEARN
  └─ innerfolder
     └─ text.txt
        node_modules
        .gitignore
        basic_server.js
        command_line_get_input.js
        export.js
        filter.js
        folder_path.js
        index.js
        package-lock.json
        package.json
        simple_api_data.js
        simple_api.js

JS folder_path.js > ...
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const argv = process.argv;
5  //console.log(dirpath);
6
7  if(argv[2]=== 'add'){
8      fs.writeFileSync(`${dirpath}/${argv[3]}`, argv[4]);
9      console.log(`${argv[3]} file is created in "${dirpath}" folder`);
10 }else if(argv[2]=== 'remove'){
11     fs.unlinkSync(`${dirpath}/${argv[3]}`);
12     console.log(`${argv[3]} file is deleted from "${dirpath}" folder`);
13 }

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js add text.txt 'inner text'
text.txt file is created in "G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder" folder

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $
```

The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'LEARN' folder containing 'innerfolder', 'node_modules', '.gitignore', and several JavaScript files. The code editor shows the content of 'folder_paths.js', which uses the 'fs' and 'path' modules to create or delete a file in the 'innerfolder' directory based on command-line arguments. The terminal at the bottom shows the execution of the script with 'add' and 'remove' commands, resulting in the creation and deletion of 'text.txt' in the 'innerfolder' directory.

```
EXPLORER
  ...
  LEARN
    innerfolder
    node_modules
    .gitignore
    JS basic_server.js
    JS command_line_get_input.js
    JS export.js
    JS filter.js
    JS folder_path.js
    JS index.js
    {} package-lock.json
    {} package.json
    JS simple_api_data.js
    JS simple_api.js

JS folder_path.js
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const argv = process.argv;
5  //console.log(dirpath);
6
7  if(argv[2]=== 'add'){
8    fs.writeFileSync(`${dirpath}/${argv[3]}`, argv[4]);
9    console.log(`${argv[3]} file is created in "${dirpath}" folder`);
10 }else if(argv[2]=== 'remove'){
11   fs.unlinkSync(`${dirpath}/${argv[3]}`);
12   console.log(`${argv[3]} file is deleted from "${dirpath}" folder`);
13 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node folder_path.js add text.txt 'inner text'
text.txt file is created in "G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder" folder

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node folder_path.js remove text.txt
text.txt file is deleted from "G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder" folder

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

Create and get file list from a inner folder of project folder: -----

The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'LEARN' folder containing 'innerfolder', 'node_modules', '.gitignore', and several JavaScript files. The code editor shows the content of 'read_dir.js', which uses the 'fs' module to create five text files in the 'innerfolder' directory and then reads the directory to log the file names. The terminal at the bottom shows the execution of the script, which creates the files and then lists them in an array and individually.

```
EXPLORER
  ...
  LEARN
    innerfolder
      textfile_0.txt
      textfile_1.txt
      textfile_2.txt
      textfile_3.txt
      textfile_4.txt
    node_modules
    .gitignore
    JS basic_server.js
    JS command_line_get_input.js
    JS export.js
    JS filter.js
    JS folder_path.js
    JS index.js
    {} package-lock.json
    {} package.json
    JS read_dir.js
    JS simple_api_data.js
    JS simple_api.js

JS read_dir.js
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4
5  for(i=0; i<5; i++){
6    fs.writeFileSync(`${dirpath}/textfile_${i}.txt`, 'text file content');
7  }
8
9  fs.readdir(dirpath, (err, files)=>{
10   console.log("show files list in console as array");
11   console.log(files);
12   console.log("get individual files name one by one");
13   files.forEach((file)=>{ console.log(file) });
14 });

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_dir.js
show files list in console as array
[
  'textfile_0.txt',
  'textfile_1.txt',
  'textfile_2.txt',
  'textfile_3.txt',
  'textfile_4.txt'
]
get individual files name one by one
textfile_0.txt
textfile_1.txt
textfile_2.txt
textfile_3.txt
textfile_4.txt

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```


EXPLORER

LEARN

innerfolder

textfile_0.txt

textfile_1.txt

textfile_2.txt

textfile_3.txt

textfile_4.txt

node_modules

.gitignore

basic_server.js

command_line_get_input.js

export.js

filter.js

folder_path.js

index.js

package-lock.json

package.json

read_dir.js

simple_api_data.js

simple_api.js

JS read_dir.js U X

JS read_dir.js > fs.readdir() callback

```
1 const fs = require('fs');
2 const path = require('path');
3 const dirpath = path.join(__dirname, 'innerfolder');
4
5 for(i=0; i<5; i++){
6   fs.writeFileSync(`${dirpath}/textfile_${i}.txt`, 'text file content')
7 }
8
9 fs.readdir(dirpath, (err, files)=>{
10   console.log("show files list in console as array");
11   console.log(files);
12   console.log("get individual files name one by one");
13   files.forEach((file)=>{ console.log(file) });
14 });
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)

● \$ node read_dir.js

○ show files list in console as array

[

'textfile_0.txt',

'textfile_1.txt',

'textfile_2.txt',

'textfile_3.txt',

'textfile_4.txt'

]

get individual files name one by one

textfile_0.txt

textfile_1.txt

textfile_2.txt

textfile_3.txt

textfile_4.txt

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)

\$